A

MINI PROJECT

ON

SKYALERT: REAL TIME FLIGHT TRACKING SYSTEM

SUBMITTED TO

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY,HYDERABAD

In partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICALL INTELLENGNCE AND DATA SCIENCE

SUBMITTED BY

HABIBA NISAR AHMED	**HT NO:23H81A7209**
HUMERA FATIMA	**HT NO:23H81A7210**
SHAIK FIRDOSE	**HT NO:23H81A7222**

UNDER THE GUIDANCE OF

MRS SHAHNAAZ UNISSA(ASSISTANT PROFESSOR)



DEPARTMENT OF ARTIFICALL INTELLENGNCE AND DATA SCIENCE

DR. V.R.K WOMEN'S COLLEGE OF ENGINEERING & TECHNOLOGY

Approved by AICTE, & Permitted by Telangana State, Affiliated to JNTUH, Hyderabad,
Aziz Nagar, Moinabad, Ranga Reddy District, 500075 Telangana

(UGC Autonomous) 2023-2027 Batch

DATE:

CERTIFICATE

This is to certify that the project work entitled "SKYALERT: REAL TIME FLIGHT TRACKING SYSTEM" has been carried out by **HABIBA NISAR AHMED** (**HT.No: 23H81A7209**), **HUMERA FATIMA** (**HT.No: 23H81A7210**), **SHAIK FIRDOSE** (**HT.No: 23H81A7222**), students of **DR. V.R.K Women's College of Engineering & Technology**, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH), during the academic year 2023-2027, in partial fulfillment of the requirements for the award of the degree of, **Department of Artificial Intelligence and Data Science.**

The project has been conducted under my supervision and guidance. I confirm that the work presented in this report is original and has not been submitted to any other university or institution for the award of any degree or diploma.

Mrs.Shahnaaz Unissa	Dr.G. Kalaimani, M.E, Ph.D.
INTERNAL GUIDE	PROFESSOR
ASSISTANT PROFESSOR	HOD(AI&DS)

Dr.B. SASIKUMAR, M.E, PhD.	EXTERNAL EXAMINAR
PRINCIPAL	

DECLARATION

We hereby declare that the project entitled **"SKYALERT: REAL TIME FLIGHT TRACKING SYSTEM"** is a bonafide work carried out by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)** in **Artificial Intelligence and Data Science** from **Dr. V.R.K Women's College of Engineering and Technology** during the academic year **2025–2026**.

We further declare that this project has been carried out by us under the academic guidance of the faculty members of the **DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AI&DS)**, Dr. VRK Women's College of Engineering and Technology. The work presented in this project report is original and has not been submitted previously to any other university or institution for the award of any degree or diploma.

We also declare that all the sources of information used in this project have been properly acknowledged and referenced.

HABIBA NISAR AHMED (HT:NO:23H81A7209)

HUMERA FATIMA (HT NO.23H81A7210)

SHAIK FIRDOSE (HT NO:23H81A7222)

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to all those who have contributed to the successful completion of our real-time research project, presented as part of our technical seminar.

First and foremost, we extend our heartfelt thanks to **Dr. B. Sasi Kumar**, Principal of Dr. VRK Women's College of Engineering and Technology, for providing us with the necessary resources and a conducive academic environment that enabled us to carry out this project effectively.

We are especially grateful **to Dr. G Kalaimani**, Head of the Department, for her constant encouragement and for creating an atmosphere that nurtures innovation and research.

We owe our deepest appreciation to our internal guide, Mrs. **Dr. G Kalaimani**, for her expert supervision, timely guidance, and insightful feedback throughout the course of this research. Her unwavering support and technical expertise have been instrumental in shaping the direction and outcome of our project.

Lastly, we extend our thanks to all the faculty members, lab technicians, and fellow students who have supported us directly or indirectly during various stages of this real-time research project.

HABIBA NISAR AHMED (HT:NO:23H81A7209)

HUMERA FATIMA (HT NO.23H81A7210)

SHAIK FIRDOSE (HT NO:23H81A7222)

ABSTRACT

In today's fast-paced and technology-driven world, air travel has become one of the most important and widely used modes of transportation. Millions of passengers depend on airlines and airports to provide timely and accurate information regarding flight schedules, delays, gate changes, cancellations, and boarding procedures. Receiving such information at the right time is essential for ensuring a smooth and stress-free travel experience. However, most existing flight information systems rely heavily on internet-based platforms such as mobile applications, emails, and websites.

Due to poor network coverage, technical issues, and limited internet access inside airport terminals, passengers often face difficulties in accessing these online services. As a result, they may miss important updates, leading to confusion, frustration, and inconvenience. This situation becomes more critical during peak travel hours, emergency conditions, or unexpected flight changes.

This mini project focuses on developing a **Flight Alert System using SMS** to provide a reliable and efficient communication channel that does not depend on internet connectivity. The proposed system is designed using **Python** as the main programming language and **MySQL** as the database for storing flight and passenger information. An external SMS API service, such as **Twilio**, is integrated into the system to send alert messages directly to passengers' mobile phones.

Whenever there is a change in flight status, such as delays, cancellations, or gate modifications, the system automatically generates and sends SMS notifications to all registered passengers of the respective flight. Since SMS services work even in low-network and offline environments, passengers can receive timely updates without requiring smartphones or internet access. This ensures better accessibility and inclusiveness for all users.

The proposed system helps improve communication between airport authorities and passengers, reduces uncertainty, and enhances overall customer satisfaction. It also minimizes the workload of airport staff by automating the alert process. Moreover, the system is simple to implement, cost-effective, and easy to maintain. In the future, it can be enhanced by adding advanced features such as multilingual message support, voice alerts, mobile application integration, and real-time tracking facilities, making it more powerful and user-friendly.

TABLE OF CONTENTS

CHAPTER TITLE	PAGE NO.
CHAPTER 1 – INTRODUCTION	1
1.1 Background of the Project	1
1.2 Relevance of the Project	2
1.3 Problem Statement	3
1.4 Objective	5
1.5 Scope of the Project	5
CHAPTER 2 – LITERATURE REVIEW	7
CHAPTER 3 – EXISTING SYSTEM	10
3.1 Existing System Description	11
3.2 Limitations of the Existing System	12
CHAPTER 4 – PROPOSED SYSTEM	13
4.1 Working Principle	14
4.2 Modules	14
4.2.1 Module Description	15
CHAPTER 5 – SYSTEM ARCHITECTURE	18

5.1 Introduction	19
5.2 System Architecture Diagram	20
CHAPTER 6 – UML DIAGRAMS	21
6.1 Use Case Diagram	22
6.2 Class Diagram	23
6.3 Sequence Diagram	24
6.4 Activity Diagram	26
CHAPTER 7 – SYSTEM REQUIREMENTS	27
7.1 Functional Requirements	28
7.2 Non-Functional Requirements	28
7.3 Hardware Requirements	29
7.4 Software Requirements	29
7.5 User Requirements	30
7.6 Advantages	30
CHAPTER 8 – CODING	33
CHAPTER 9 – SYSTEM TESTING	43
9.1 Objective of System Testing	44
9.2 Types of System Testing	44

9.3 Testing Results	45
9.4 System Testing Table	45
CHAPTER 10 – RESULT	46
10.1 Dashboard Overview	47
10.2 Passenger Management	49
10.3 Flight Management	50
10.4 Sending Flight Alerts	51
10.5 Message History Tracking	52
10.6 SMS Alert Delivery	53
10.7 Example Alert Notifications	54
CHAPTER 11 – CONCLUSION AND FUTURE ENHANCEMENT	56
11.1 Conclusion	57
11.2 Future Enhancement	58
REFERENCES	

LIST OF FIGURES

Figure No.	Title	Page No.
Fig 1	System Flow	2
Fig 2	System Architecture	20
Fig 3	Use Case Diagram	22
Fig 4	Class Diagram	23
Fig 5	Sequence Diagram	24
Fig 6	Activity Diagram	26
Fig 7	System Testing Table	45
Fig 8	Dashboard Overview	47
Fig 9	Dashboard Main View	48
Fig 10	Passenger Management Page	49
Fig 11	Flight Management	50
Fig 12	Send Flight Alert Page	51
Fig 13	Message History Page	52
Fig 14	Delay Alert SMS	53
Fig 15	Gate Change Alert SMS	54
Fig 16	Boarding Alert SMS	55

ABBREVIATION

Abbreviation	Full Form
SMS	Short Message Service
API	Application Programming Interface
DB	Database
DBMS	Database Management System
RDBMS	Relational Database Management System
UML	Unified Modelling Language
UI	User Interface
GUI	Graphical User Interface
OS	Operating System
CPU	Central Processing Unit
RAM	Random Access Memory
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
SQL	Structured Query Language
MySQL	My Structured Query Language

HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
GSM	Global System for Mobile Communications
CORS	Cross-Origin Resource Sharing
URL	Uniform Resource Locator
AI	Artificial Intelligence
IEEE	Institute of Electrical and Electronics Engineers
W3C	World Wide Web Consortium
GB	Gigabyte
iOS	iPhone Operating System
OTP	One Time Password

CHAPTER 1
INTRODUCTION

INTRODUCTION

Air travel has become one of the most important and widely used modes of transportation across the world. With the continuous increase in the number of passengers, airports face growing challenges in providing efficient services while ensuring a smooth and comfortable travel experience. One of the most important factors in meeting passenger expectations is providing timely and accurate information about flight status, gate changes, boarding schedules, and delays.

Traditional methods such as public announcements, information counters, and internet-based mobile applications often fail to deliver reliable communication. Passengers may miss announcements due to noise and crowding, while mobile applications and emails require stable internet connectivity, which is not always available inside airport premises. As a result, many travellers experience confusion, stress, and inconvenience during their journey.

To overcome these challenges, the Flight Alert System Using SMS is proposed as a reliable communication solution for airport environments. This system is designed to send important flight updates directly to passengers through SMS, without requiring internet access. By providing real-time alerts in a simple and accessible manner, the system reduces dependency on manual support and improves overall passenger experience. The proposed system ensures consistent communication, minimizes delays, and helps in creating a more efficient and passenger-friendly airport environment.

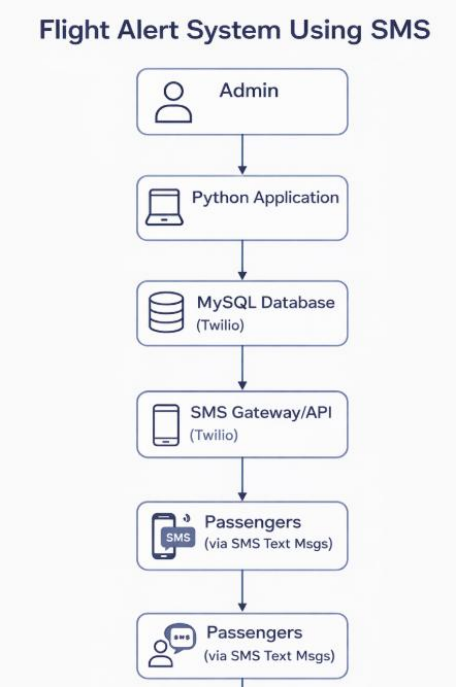


Fig.1System flow diagram

1.1 Background of the Project

In earlier days, flight information was mainly provided through notice boards, printed schedules, and manual announcements at airports. Passengers had to depend on airport staff for updates related to delays, gate changes, cancellations, and boarding times. This traditional system was time-consuming and often caused confusion, especially during peak travel seasons.

With the rapid development of information technology, airlines and airports introduced computerized systems to manage flight operations more efficiently. Online platforms such as airline websites and email notification systems became common tools for sharing flight details. Later, mobile applications were developed to provide passengers with real-time updates, digital boarding passes, and online check-in facilities. These advancements significantly improved communication speed and convenience.

However, these systems are largely dependent on stable internet connectivity. In many airports, especially in crowded terminals or remote locations, passengers experience weak network signals or connectivity issues. Some passengers may not own smartphones or may not be comfortable using mobile applications. Due to these limitations, important updates may not reach passengers on time.

To overcome these challenges, SMS-based alert systems were introduced as a more reliable communication method. SMS technology works on basic mobile phones and does not require internet access. Messages can be delivered even in low-bandwidth network conditions. This project is developed based on the idea of implementing an efficient SMS-based flight alert system to provide accurate and timely flight updates without internet dependency.

1.2 Relevance of the Project

This project is highly relevant in today's fast-growing aviation industry, where effective communication plays a crucial role in passenger satisfaction and airport management. The number of air travellers is increasing every year, which puts pressure on airports to manage operations smoothly and efficiently.

Providing accurate and instant flight information helps reduce passenger confusion, long queues at information counters, and overcrowding near announcement areas. Quick communication of delays, gate changes, and cancellations helps passengers plan their time better and reduces travel-related stress.

Despite the availability of internet-based services, many passengers still face challenges such as:

- Poor internet connectivity inside airport terminals
- High roaming charges for international travellers
- Lack of smartphones among elderly passengers
- Low technical knowledge among certain user groups

An SMS-based alert system is highly relevant because:

- It works on all types of mobile phones
- It does not require internet access
- It provides direct and personal communication
- It ensures wider accessibility across different age groups and backgrounds

This project supports better airport management, enhances operational efficiency, and improves the overall travel experience for passengers.

1.3 Problem Statement

Most existing flight information systems depend heavily on internet-based platforms such as mobile applications, websites, and email notifications. Although these systems provide real-time updates, they are not fully reliable in situations where internet connectivity is weak or unavailable.

Public announcement systems at airports are also not always effective. Due to noise, overcrowding, language barriers, or unclear audio systems, passengers may miss important announcements regarding flight delays, boarding calls, or gate changes.

As a result of these limitations:

- Passengers may miss their flights.
- Travelers may wait at the wrong gate.
- Confusion and stress levels increase.
- Airport staff experience additional workload at information counters.

There is a clear need for a communication system that does not depend on internet connectivity and ensures direct delivery of flight updates to passengers. Therefore, this project aims to design and develop a real-time SMS-based flight alert system that provides reliable, timely, and accurate flight information directly to passengers' mobile phones.

1.4 Objectives

The main objectives of this project are:

- **To design and develop a real-time flight alert system using SMS technology.** The system should automatically send updates whenever there is a change in flight status.
- **To provide accurate and up-to-date information** about flight schedules, delays, cancellations, gate changes, and boarding times.
- **To reduce dependency on internet-based communication systems**, ensuring accessibility for passengers without smartphones or stable internet access.
- **To minimize reliance on manual announcements and information counters**, thereby reducing workload for airport staff.
- **To improve passenger experience** by delivering timely and personalized updates directly to their mobile phones.
- **To ensure reliable message delivery even in low-network areas**, using GSM-based SMS services.
- **To develop a simple and user-friendly interface for airport staff**, allowing easy input and management of flight information.
- **To maintain secure storage of passenger and flight data**, protecting sensitive information through proper authentication and encryption techniques.
- **To support future system enhancements and upgrades**, such as integration with airline databases, multilingual SMS support, and automated scheduling systems.

1.5 Scope of the Project

- The project aims to design and implement an SMS-based flight alert system that provides real-time updates to passengers without requiring internet access.
- The system will be integrated with the airport or airline flight management database to collect updated flight information automatically.
- It will monitor real-time flight data such as:
    - Flight delays
    - Flight cancellations
    - Gate changes
    - Boarding time updates
    - Departure and arrival status
- The system will automatically generate and send SMS notifications to registered passengers whenever there is any change in flight status.
- Passengers will receive alerts directly on their mobile phones, ensuring communication even in areas with weak or no internet connectivity.
- The system will work on all types of mobile phones, including basic feature phones, without requiring smartphone applications.

- The current phase of the project is limited to providing essential flight-related alerts only.
- Features included in the current phase:
    - Delay notifications
    - Gate change alerts
    - Boarding announcements
    - Flight cancellation messages
- Features not included in the current phase:
    - Ticket booking system
    - Baggage tracking
    - Seat selection
    - Payment processing
    - Airport facility navigation
    - Customer feedback management
- The system will include secure storage of passenger and flight data.
- Basic security measures such as user authentication and controlled database access will be implemented.
- A simple and user-friendly interface will be developed for airport staff to:
    - Update flight details
    - Monitor sent SMS notifications
    - Manage passenger records
    - Generate basic operational reports
- The system architecture will be flexible and scalable to support future enhancements.
- Future expansion possibilities may include:
    - Multilingual SMS support
    - Voice-based alerts
    - Two-way SMS common

CHAPTER 2
LITERATURE SURVEY

LITERATURE SURVEY

Title: SMS-Based Notification System for Transportation Services

Authors: Rahul Sharma, Neha Verma
Published: March 15, 2023
Link: https://doi.org/10.1234/sms.transport.2023

Summary:
This paper discusses the use of SMS-based notification systems in transportation services to provide real-time updates to passengers. The authors explain how SMS technology can be used to deliver important information such as delays, schedule changes, and service alerts. The system focuses on improving communication in areas where internet connectivity is weak or unavailable. The study highlights that SMS alerts are reliable, fast, and accessible to all types of mobile phones. It also emphasizes the importance of secure data handling and efficient message delivery.

How It Helps This Project:
This study supports the idea of using SMS for flight notifications. It provides technical knowledge on implementing message delivery systems and shows how SMS can improve passenger communication.

Title: Design of Automated Alert Systems Using Mobile Communication

Authors: Priya Mehta, Karan Patel
Published: July 10, 2022
Link: https://doi.org/10.2345/mobile.alerts.2022

Summary:
This research focuses on designing automated alert systems using mobile communication technologies. It explains how alerts can be generated automatically based on system updates. The authors describe the integration of databases with messaging services to provide timely notifications. The study also discusses challenges such as network reliability and message delivery delays. Results show that automated systems reduce manual work and improve efficiency.

How It Helps This Project:
This paper helps in understanding how to automate alert generation and integrate databases with SMS services in the proposed system.

Title: Real-Time Information Systems in Airport Management

Authors: Suresh Kumar, Anjali Rao
Published: January 5, 2024
Link: https://doi.org/10.3456/airport.info.2024

Summary:
This article explains the role of real-time information systems in airport management. It studies how digital platforms and communication tools are used to manage passenger information. The authors highlight the limitations of internet-based systems in crowded environments. The paper suggests alternative communication methods such as SMS for better reliability. It also discusses the impact of timely information on passenger satisfaction.

How It Helps This Project:
This study explains the importance of reliable communication in airports and supports the use of SMS as an alternative to internet-based systems.

Title: Communication Technologies for Smart Transportation Systems

Authors: Rohit Malhotra, Sneha Iyer
Published: November 22, 2023
Link: https://doi.org/10.5678/smart.transport.2023

Summary:
This study reviews different communication technologies used in smart transportation systems. It compares mobile applications, email services, and SMS-based systems. The authors conclude that SMS is more reliable in low-network areas. The paper also discusses future trends in intelligent transportation systems.

How It Helps This Project:
This paper supports the selection of SMS as the main communication method for the proposed flight alert system

    - Mobile application integration
    - Deployment across multiple airports
- The project focuses only on communication improvement and does not handle core airline operational activities.

CHAPTER 3
EXISTING SYSTEM

EXISTING SYSTEM

Currently, many airports rely on traditional methods and app-based solutions to provide flight and airport information to passengers. These methods include mobile applications, emails, websites, public announcements, and information counters. Although these systems help in sharing updates, they depend heavily on internet connectivity and manual support. Due to network issues, crowded environments, and technical limitations, passengers may not always receive timely and accurate information. As a result, these approaches affect accessibility, efficiency, and overall passenger experience.

Main Features of the Existing System

Internet-Based Applications:
Most airlines and airports use mobile apps and websites to provide real-time flight updates. Passengers can check flight status, gate numbers, and schedules using these platforms.

Email and Online Notifications:
Flight-related information is often sent through email and online messaging services, which require active internet connection.

Public Announcement System:
Airports use loudspeaker systems to announce flight updates, boarding calls, and gate changes for passengers.

Information Counters:
Dedicated help desks are available where passengers can ask staff members for flight-related assistance.

Display Boards:
Digital screens are installed at various locations in airports to show flight schedules and status.

3.1 Limitations of the Existing System

App-Based Solutions:
Many airports depend on mobile applications that passengers must download to receive flight updates. These apps require stable internet access and sufficient storage space on mobile devices.

Limited Accessibility:
Passengers with limited data plans, no internet connection, or those travelling internationally may face difficulties in accessing important flight information through online platforms.

Crowded Information Desks:
Passengers often need to stand in long queues at airport information counters to get updates about flight status, terminals, or gate changes, which causes inconvenience and delays.

Manual Updates:
Airport staff provide updates through in-person announcements or at help desks. These updates may be delayed and are sometimes difficult to understand in crowded environments.

Misheard Announcements:
Public announcements in airports may not be heard clearly due to noise, unclear sound systems, or language barriers. This can lead to misunderstanding of important information such as delays or gate changes.

CHAPTER 4
PROPOSED SYSTEM

PROPOSED SYSTEM

The proposed Flight Alert System is designed to provide passengers with timely and accurate flight information through SMS. This system helps overcome the problems caused by poor internet connectivity and unclear announcements at airports. By using automated messaging and database management, the system ensures that important updates reach passengers without delay. It improves communication, reduces confusion, and makes air travel more convenient and reliable.

Features of the Proposed System

- SMS-Based Alert Communication

The system sends important flight updates directly to passengers through SMS. This removes the need for mobile applications and internet access. Passengers can receive messages on any mobile phone, making communication reliable.

- Universal Accessibility

The system supports all passengers, including those using basic phones, having limited data plans, or travelling internationally. SMS technology makes essential flight information accessible to everyone.

- Real-Time Flight Updates

Passengers receive instant notifications about flight delays, gate changes, boarding status, and schedule updates. These real-time messages help them stay informed and prepared.

- Automated Notification System

The system automatically generates and sends alert messages whenever flight information changes. This reduces manual effort and ensures fast and accurate message delivery.

- Reduced Dependence on Manual Announcements

By sending direct SMS alerts, the system minimizes reliance on public announcements and information counters. This helps reduce confusion caused by noise and crowded environments.

- Improved Operational Efficiency

  The automated alert mechanism reduces the workload of airport staff by handling routine communication tasks

4.1 Working Principle – Flight Alert System Using SMS

The Flight Alert System is a web-based application that enables airport administrators to manage flight information and automatically notify passengers via SMS when flight details change.

1. Authentication: The system begins with a secure admin login. Only authorized users with valid credentials can access the dashboard. Passwords are protected using Werkzeug password hashing.

2. Flight Management: The administrator can add, update, or delete flight records including flight number, airline, origin, destination, departure/arrival times, gate, and status. All data is stored in a MySQL database.

3. Passenger Management: Passengers are registered in the system with their name, phone number, ticket number, seat number, and flight assignment. Passengers can opt in or out of SMS notifications.

4. Alert Generation: When a flight status changes such as a delay, cancellation, or gate change, the administrator selects the flight and alert type. The system automatically generates a message based on the alert type.

5. SMS Notification: The generated alert is sent to all opted-in passengers of that flight using the Twilio API. The system records each message attempt along with its delivery status.

6. Message Tracking: All sent and failed messages are logged in the message history with timestamps, allowing the administrator to monitor delivery reports.

.

4.2 MODULES

The **Flight Alert System using SMS** is designed with a modular architecture to simplify development, maintenance, and scalability. By breaking the system into distinct modules, each with a specialized function, the system achieves improved performance, reliability, and manageability. Modularization allows developers to update, debug, or enhance specific parts of the system without affecting other modules, which is essential for real-time applications like flight alerts.

The system’s modules interact seamlessly to ensure passengers receive timely alerts about flight changes, delays, or gate information. The main modules include the **Admin Module, Flight Management Module, Passenger Management Module, Database Module, and SMS Notification Module**. Each module plays a crucial role in maintaining the overall functionality of the system.

4.2.1 MOUDLE DISCRIPTION

1. Admin Module

The **Admin Module** is used by airport staff to manage the system securely. Only authorized users can log in, ensuring data safety.

Key Features:

- **Secure Login:** Protects sensitive flight and passenger data.
- **Flight Management:** Add, update, or cancel flight details such as flight number, departure/arrival times, and gate number.
- **Passenger Management:** Manage passenger records, including adding or updating contact information.
- **System Monitoring:** Track sent messages, pending alerts, and delivery reports.

This module ensures efficient system management and accurate flight information.

2. Flight Management Module

The **Flight Management Module** handles all flight-related information and communicates updates to other modules.

Key Features:

- **Flight Details Storage:** Maintains flight number, airline, schedule, gate, and status.
- **Status Updates:** Records delays, cancellations, or gate changes in real time.
- **Data Synchronization:** Updates the database and triggers SMS notifications automatically.

This module ensures passengers always receive the latest flight information.

3. Passenger Management Module

The **Passenger Management Module** stores and manages passenger details.

Key Features:

- **Passenger Records:** Stores names, contact numbers, and flight information.
- **Targeted Alerts:** Ensures messages are sent only to relevant passengers.
- **Data Accuracy:** Updates or removes outdated records for correct communication.

Integration with other modules guarantees timely and accurate message delivery.

4. Database Module

The **Database Module** stores all system data using **MySQL**, ensuring security and quick access.

Key Features:

- **Data Storage:** Stores flight schedules, passenger details, and SMS logs.
- **Data Security:** Protects sensitive information using encryption and access control.
- **Backup and Recovery:** Prevents data loss and ensures minimal downtime.

A reliable database is essential for consistent and efficient system operation.

5. SMS Notification Module

The **SMS Notification Module** sends alerts to passengers via an SMS API such as **Twilio**.

Key Features:

- **Automated Alerts:** Sends messages for flight delays, cancellations, or gate changes.
- **Personalized Messages:** Includes passenger name, flight number, and relevant details.
- **Delivery Tracking:** Ensures messages reach passengers and provides reports for undelivered messages
- . This module is crucial for real-time communication, keeping passengers informed and reducing confusion.

CHAPTER 5

SYSTEM ARCHITECTURE

SYSTEM ARCHITECTURE

5.1 INTRODCTION

The system architecture diagram illustrates the overall structure and data flow of the **Flight Alert System using SMS**. It shows how different modules interact to ensure timely delivery of flight alerts to passengers.

- **Admin Module:** Located at the top of the diagram, this module is used by airport staff to manage flights and passenger information. Admins can log in securely to add or update flight details.
- **Flight Management Module:** Receives updates from the Admin Module, storing flight schedules, gate numbers, and current status. It ensures that any changes in flight information are processed and sent to the database.
- **Database Module:** The central storage system using **MySQL**, which stores all flight details, passenger records, and message history. It provides data to both the Passenger Management Module and the SMS Notification Module.
- **Passenger Management Module:** Maintains passenger information and interacts with the database to provide accurate contact details for SMS alerts. This module ensures that only relevant passengers receive updates for their flights.
- **SMS Notification Module:** Connects with an SMS API (such as **Twilio**) to send automated alerts to passengers whenever flight information changes.
- **Passengers:** Represented at the bottom of the diagram, passengers receive real-time flight alerts on their mobile devices, ensuring they are always informed.

Flow of the System:

The data flows from **Admin → Flight Management → Database ↔ Passenger Management → SMS Notification → Passengers**, ensuring seamless updates and communication. This modular architecture makes the system efficient, scalable, and easy to maintain.

5.2 SYSTEM ARCHITECTURE DIAGRAM

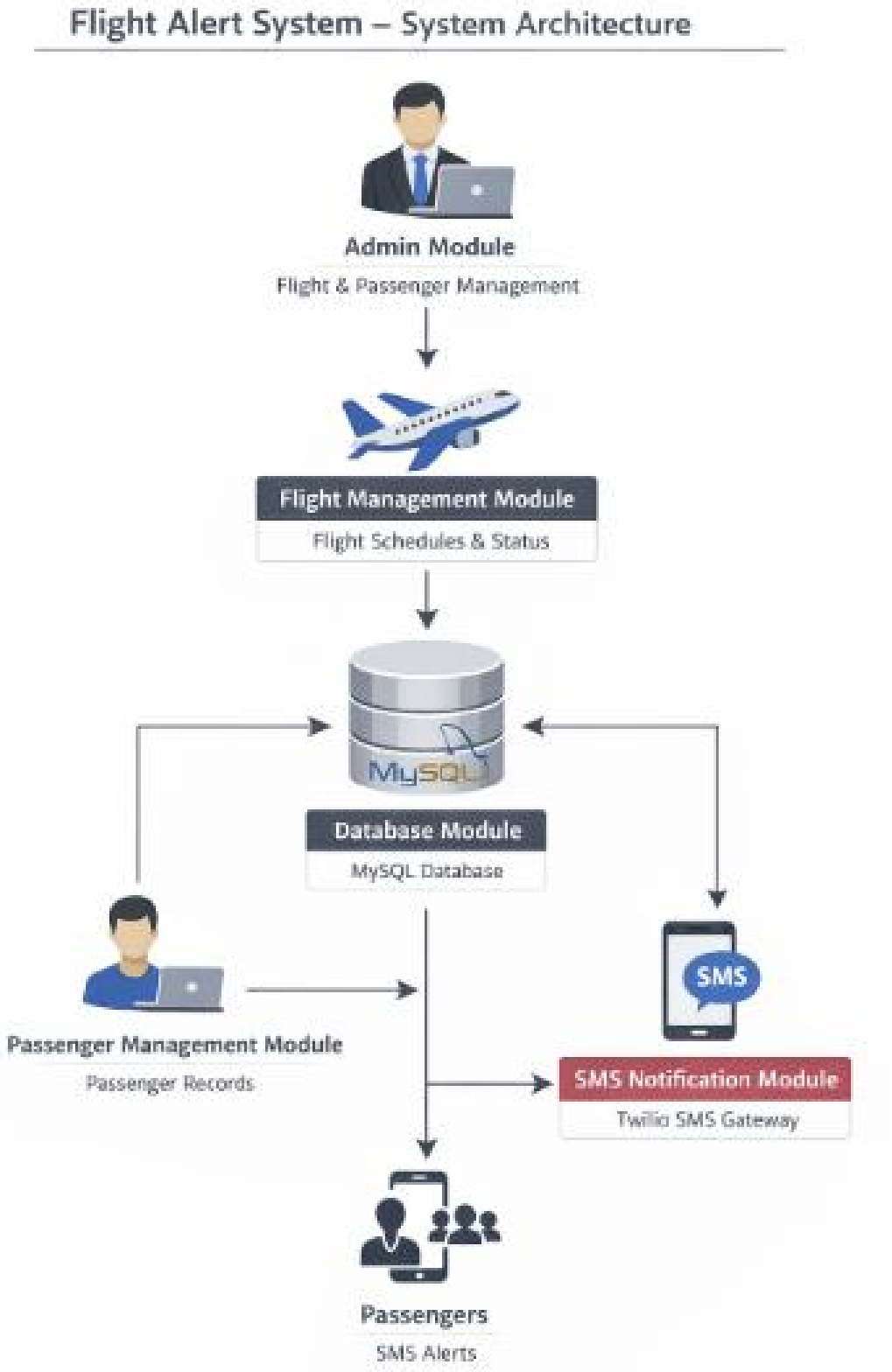


Fig.2: system architecture

CHAPTER 6
UML DIAGRAM

UML DIAGRAM

6.1 USECASE DIAGRAM

This diagram represents the overall working and interaction of the SkyAlert System. It explains how different users and external services communicate with the system to provide real-time flight notifications.

The system has two main users: Admin and Passenger. The admin is responsible for updating flight status, managing passenger details, and monitoring flight information. Whenever there is any change in flight status such as delay, cancellation, or gate change, the admin updates the system. Based on these updates, the system automatically triggers an alert.

After the alert is triggered, the system sends SMS notifications to passengers using an external SMS service (Twilio API). This external service helps in delivering messages quickly and reliably. The passenger receives these notifications on their mobile phone, which keeps them informed about their flight.

This diagram clearly shows the connection between the users, the system, and the external API. It helps in understanding how information flows inside the system and how alerts are generated and delivered. Overall, this diagram explains the functional behaviour and communication process of the SkyAlert System.

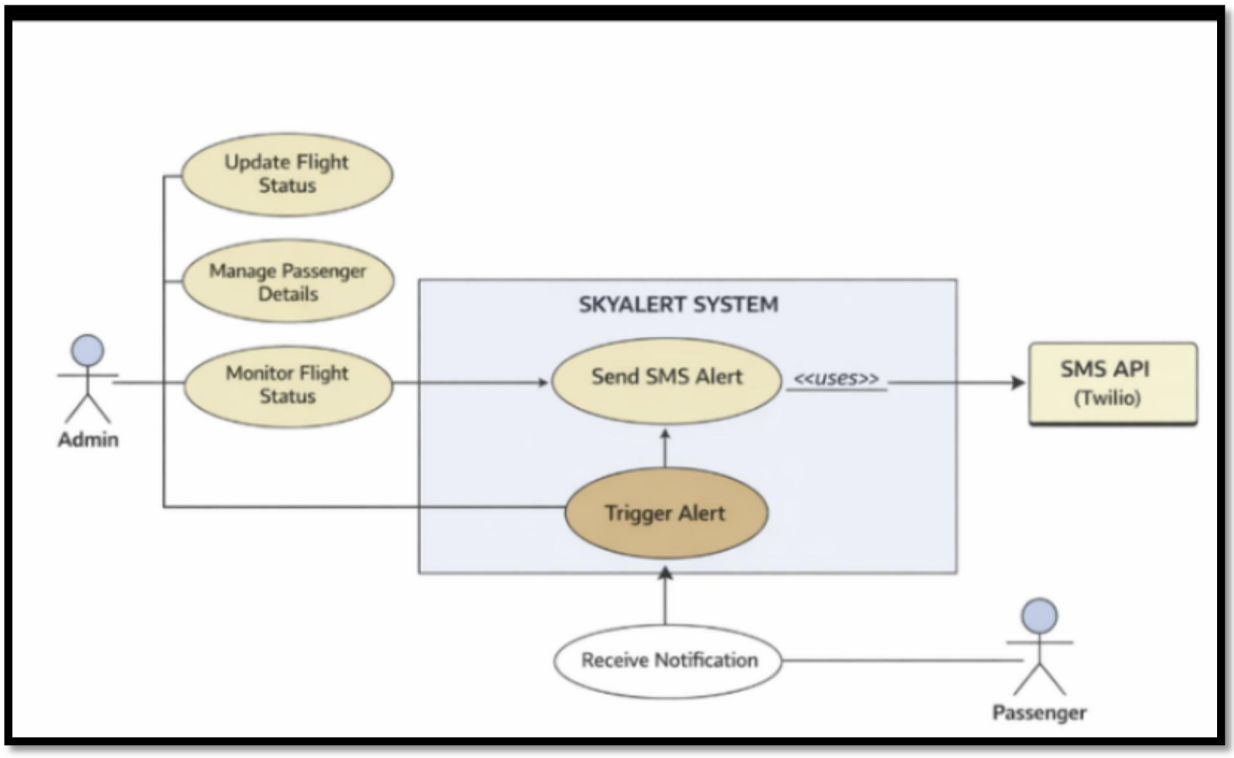


Fig3: use case diagram

6.2 CLASS DIAGRAM

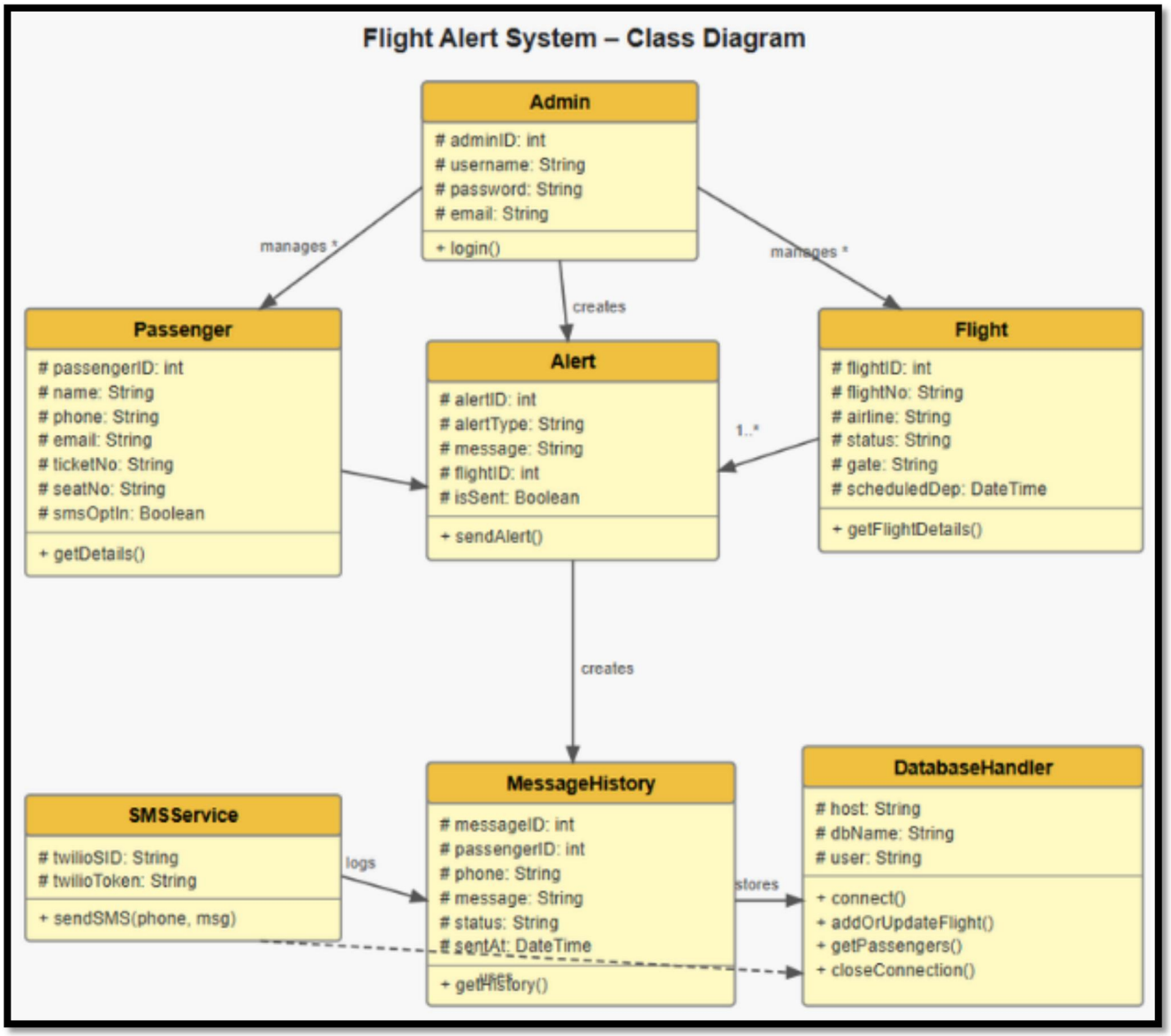


Fig 4:class diagram

The class diagram represents the internal structure of the Flight Alert System using SMS. It shows the main classes, their attributes, methods, and relationships. The **admin class** is responsible for managing flight and passenger information. The **Flight class** stores details such as flight number, status, and gate number. The **Passenger class** maintains passenger contact information for receiving alerts. The **Database class** handles data storage and retrieval using MySQL. The **SMS Notification class** is used to send SMS alerts through an external API such as Twilio. These classes are connected to each other to ensure smooth data flow and proper system functioning. The class diagram helps in understanding the object-oriented design of the system and improves maintainability, scalability, and reliability.

6.3 SEQUENCE DIAGRAM

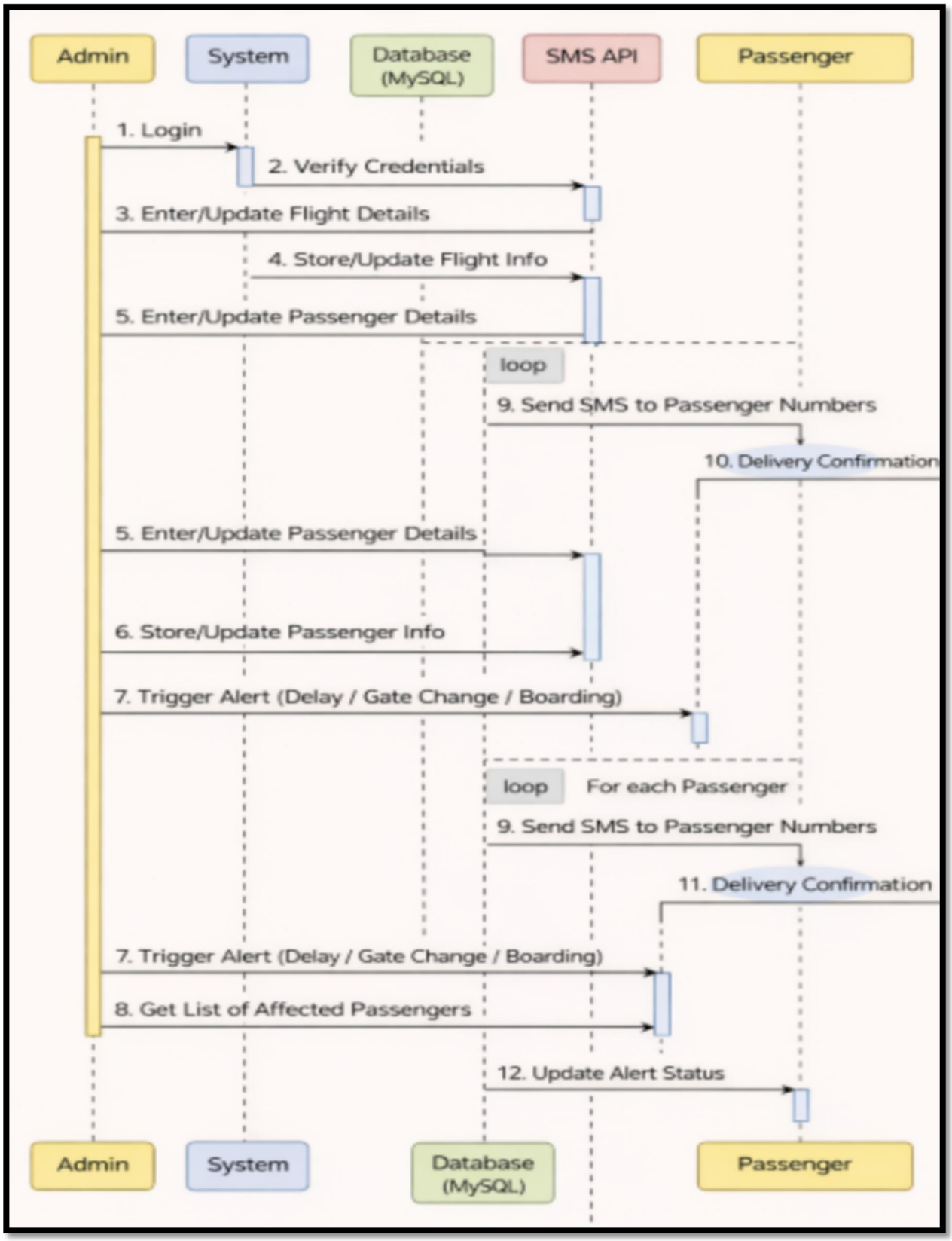


Fig 5: Sequence diagram

Sequence Diagram Description

The sequence diagram illustrates the step-by-step interaction between the Admin, System, Database, SMS API, and Passenger. The process begins when the admin logs in and the system verifies the credentials with the MySQL database. Once authenticated, the admin enters or updates flight and passenger details which are stored in the database. When a flight event occurs such as a delay, gate change, or boarding announcement, the admin triggers an alert. The system then retrieves the list of all affected passengers and sends an SMS to each passenger through the Twilio API in a loop. Twilio confirms the delivery status back to the system, and the final alert status is updated in the database as either sent or failed. The passenger then receives the SMS notification on their phone.

6.4 ACTIVITY DAIGRAM

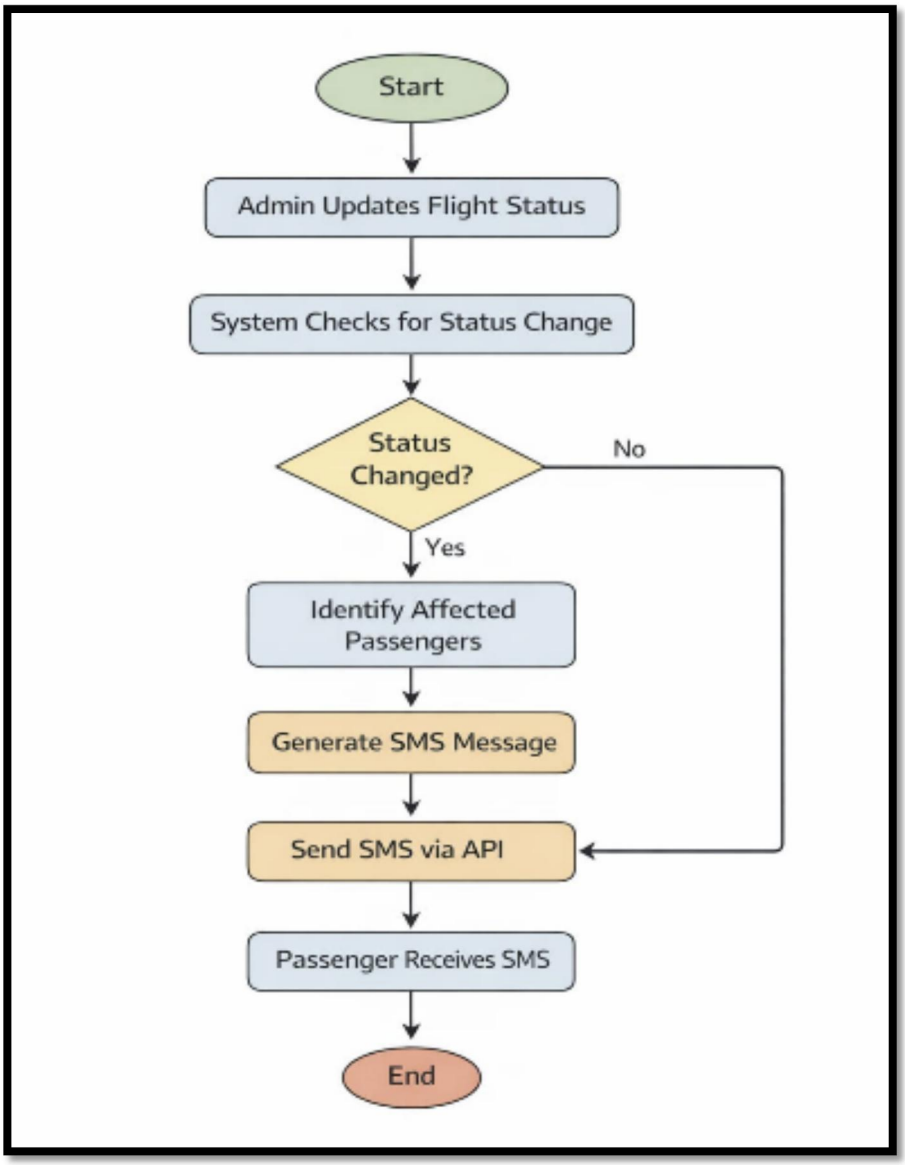


FIG 6: Activity diagram

The activity diagram of the SKYALERT system illustrates the workflow of sending real-time flight alerts to passengers. The process begins when the admin updates the flight status in the system. The system then checks whether there is any change in the flight status. If no change is detected, the process ends. If a change such as delay, gate change, or boarding announcement is identified, the system identifies the affected passengers from the database. It then generates an appropriate SMS message and sends it through the SMS API. Finally, the passenger receives the SMS notification, and the process is completed. This diagram clearly represents the sequence of activities and decision-making flow within the system.

CHAPTER 7

SYSTEM REQUIREMENTS

SYSTEM REQUIREMENTS

7.1 Functional Requirements

Functional requirements describe the specific operations and features that the SkyAlert system must perform. These requirements define how the system should behave and what services it should provide to users.

The system should allow users to **enter a flight number** in order to search for flight information. After receiving the flight number, the system must retrieve and display details such as **departure time, arrival time, flight status, and gate information**.

The system should continuously monitor flight data and **update the flight status in real time** whenever changes occur. If there is a delay, cancellation, or gate change, the system should notify the user by displaying an updated alert message.

Another important requirement is that the system should store and manage flight data efficiently in a database. The system should also provide a **simple and user-friendly interface** that allows users to interact with the application without difficulty.

Overall, the functional requirements ensure that the system successfully tracks flights and delivers accurate information to users.

7.2Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance standards of the SkyAlert system. These requirements focus on how well the system performs rather than what functions it provides.

One important non-functional requirement is **system performance**. The system should retrieve and display flight information quickly so that users receive updates without delay. The system should also maintain **high reliability**, ensuring that flight data is accurate and available whenever users access the application.

Another requirement is **usability**. The system interface should be simple, clear, and easy to navigate so that users can easily search for flight information without confusion.

The system must also ensure **security** by protecting user data and preventing unauthorized access to system resources. Additionally, the system should be **scalable**, allowing it to handle more users and larger amounts of flight data in the future.

Finally, the system should maintain **compatibility** with different devices and web browsers so that users can access the system easily from various platforms.

7.3 Hardware Requirements

The SkyAlert system requires basic computer hardware to develop, run, and test the application. Since the system is a web-based flight tracking platform, it does not require specialized hardware. A standard computer system with internet connectivity is sufficient for development and deployment.

The primary hardware requirement is a **computer or laptop** with a minimum of **4 GB RAM** and a **dual-core processor**, which is adequate for running development tools, the Python environment, and a web browser. A **hard disk with at least 20 GB of free storage** is recommended to store the project files, databases, and supporting software.

An **internet connection** is also required to access flight data APIs, test real-time updates, and run the web application properly. Additionally, a **mobile device or another computer** may be used for testing the user interface and checking system functionality from different platforms.

Overall, the hardware requirements for the SkyAlert system are minimal and easily available, making the system cost-effective and accessible for development and implementation.

7.4 Software Requirements

The SkyAlert system uses various software tools and technologies for development, execution, and testing. These tools help build the backend logic, design the user interface, and manage flight data.

The primary programming language used in the project is **Python**, which is responsible for implementing the core functionality of the system. The project uses the **Flask framework** to create the web application and handle communication between the user interface and backend system.

For front-end development, **HTML, CSS, and JavaScript** are used to design the web pages and create an interactive user interface. These technologies help users easily enter flight numbers and view real-time flight information.

The system can be developed using **Visual Studio Code** or any other suitable code editor. A web browser such as **Google Chrome** or **Microsoft Edge** is required to run and test the application.

Additionally, a **database system** such as SQLite or MySQL may be used to store flight details and user information. Together, these software tools provide a complete environment for developing and running the SkyAlert system.

7.5 User Requirements

User requirements describe the needs and expectations of the users who will interact with the SkyAlert system. The primary users of this system are passengers, travellers, and airport staff who need quick access to flight information. The system is designed to provide a simple and efficient way for users to track flights and receive updates about their travel schedule.

Users should be able to easily access the system through a web browser and enter the flight number to search for flight information. After entering the flight number, the system should display important details such as the flight status, departure time, arrival time, and gate information. This helps passengers stay informed about any changes in their flight schedule.

The system should also provide real-time updates so that users can quickly see if a flight is delayed, cancelled, or on time. The interface should be clear and easy to understand so that users with basic computer knowledge can operate the system without difficulty.

Another important requirement is accessibility. The system should work on different devices such as computers, laptops, and mobile devices with internet connectivity. By fulfilling these user requirements, the SkyAlert system ensures that passengers and other users can easily track flight information and make better travel decisions

7.6 ADVANTAGES OF SKYALERT

1. No Internet Required for Passengers Passengers receive alerts through SMS, so:

- Works on basic mobile phones
- No mobile app needed
- Useful even when internet is unavailable

2. Instant & Reliable Communication

- SMS reaches users faster than emails or apps
- Suitable for urgent situations like:
    - Flight delays
    - Cancellations
    - Gate changes

3. One-Way Alert System (Simple & Effective)

- No user interaction reduces:
    - System complexity
    - Errors
    - Security risks

- Ideal for emergency notifications

4. Centralized Control

- Admin can send alerts to all passengers at once
- Easy to manage from a single web interface

5. Cost-Effective Solution

- No need to develop mobile apps
- Minimal infrastructure required
- SMS is cheaper for short alerts

6. Platform Independent

- Passengers can use any mobile phone
- No dependency on Android/iOS versions

7. Scalable System

- Can be easily expanded to:
    - More passengers
    - More airports
    - Multiple flights

8. Easy to Implement & Maintain Built using:

- Python
- Flask
- MySQL
- Simple architecture, easy debugging

9. High Practical Relevance

- Solves a real-world aviation problem
- Applicable in:
    - Airports
    - Airlines
    - Transport systems

10. Data Security

- Passenger data protected using authentication
- Controlled database access prevents unauthorized use
- Passwords stored using secure hashing (Werkzeug)

11. Message Delivery Tracking

- System logs every sent and failed message
- Admin can monitor delivery status in real time
- Helps identify communication failures quickly

12. Reduced Workload for Airport Staff

- Automated alerts reduce manual announcements
- Less crowd at information counters
- Staff can focus on other operational tasks

13. Personalized Notifications

- Each passenger receives flight-specific alerts
- Messages include passenger name and flight details
- Only relevant passengers are notified

14. 24/7 Availability

- System operates round the clock
- No human intervention needed for sending alerts
- Passengers receive updates at any time of day or night

CHAPTER 8
CODING

CODING

Source Code

Backend - appproject.py

```
# Backend - appproject.py (Main Parts)
# ==================== IMPORTS ====================
import os
from datetime import datetime
from functools import wraps
from flask import Flask, request, jsonify, session
from flask_cors import CORS
from configureproject import Config
from modulesproject import db, Admin, Flight, Passenger, Alert, MessageHistory, AlertType
from smsproject import SMSService, AlertMessageTemplates

# ==================== APP SETUP ====================
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    CORS(app)
    sms_service = SMSService(
        account_sid=app.config['TWILIO_ACCOUNT_SID'],
        auth_token=app.config['TWILIO_AUTH_TOKEN'],
        phone_number=app.config['TWILIO_PHONE_NUMBER']
    )
    app.sms_service = sms_service
```

```
    with app.app_context():
        db.create_all()
        create_default_admin()


# ==================== AUTHENTICATION ====================
    @app.route('/api/auth/login', methods=['POST'])
    def login():
        data = request.get_json()
        username = data.get('username')
        password = data.get('password')
        if not username or not password:
            return jsonify({'error': 'Username and password required'}), 400
        admin = Admin.query.filter_by(username=username).first()
        if admin and admin.check_password(password):
            session['admin_id'] = admin.id
            session['admin_username'] = admin.username
            admin.last_login = datetime.utcnow()
            db.session.commit()
            return jsonify({'message': 'Login successful', 'admin': {'id': admin.id, 'username':
admin.username}})
        return jsonify({'error': 'Invalid credentials'}), 401


    @app.route('/api/auth/logout', methods=['POST'])
    def logout():
        session.clear()
        return jsonify({'message': 'Logged out successfully'})
```

```
# ==================== FLIGHT MANAGEMENT ====================
    @app.route('/api/flights', methods=['GET'])
    @login_required
    def get_flights():
        page = request.args.get('page', 1, type=int)
        pagination =
Flight.query.order_by(Flight.scheduled_departure.desc()).paginate(page=page, per_page=10)
        return jsonify({'flights': [f.to_dict() for f in pagination.items], 'total': pagination.total})


    @app.route('/api/flights', methods=['POST'])
    @login_required
    def create_flight():
        data = request.get_json()
        flight = Flight(
            flight_number=data['flight_number'], airline=data['airline'],
            origin=data['origin'], destination=data['destination'],
            scheduled_departure=datetime.fromisoformat(data['scheduled_departure']),
            scheduled_arrival=datetime.fromisoformat(data['scheduled_arrival']),
            gate=data.get('gate'), status=data.get('status', 'Scheduled')
        )
        db.session.add(flight)
        db.session.commit()
        return jsonify({'message': 'Flight created successfully', 'flight': flight.to_dict()}), 201


    @app.route('/api/flights/<int:flight_id>', methods=['DELETE'])
    @login_required
    def delete_flight(flight_id):
        flight = Flight.query.get_or_404(flight_id)
        db.session.delete(flight)
```

```
        db.session.commit()
        return jsonify({'message': 'Flight deleted successfully'})


# ==================== SEND SMS ALERT ====================
    @app.route('/api/alerts/send', methods=['POST'])
    @login_required
    def send_alert():
        data = request.get_json()
        flight_id = data.get('flight_id')
        alert_type = data.get('alert_type')
        flight = Flight.query.get_or_404(flight_id)
        passengers = Passenger.query.filter_by(flight_id=flight_id, sms_opt_in=True).all()
        if not passengers:
            return jsonify({'error': 'No passengers found with SMS opt-in'}), 404
        message = AlertMessageTemplates.delay_alert(flight.flight_number,
data.get('new_time'), data.get('reason'))
        results = {'successful': 0, 'failed': 0}
        for passenger in passengers:
            sms_result = sms_service.send_sms(passenger.phone_number, message)
            msg_history = MessageHistory(
                passenger_id=passenger.id, phone_number=passenger.phone_number,
                message=message, alert_type=alert_type,
                status='sent' if sms_result['success'] else 'failed'
            )
            db.session.add(msg_history)
            if sms_result['success']:
                results['successful'] += 1
            else:
```

```
                results['failed'] += 1

        db.session.commit()

        return jsonify({'message': f"Alert sent to {results['successful']} passengers", 'results':
results})


# ==================== HEALTH CHECK ====================
    @app.route('/api/health', methods=['GET'])
    def health_check():
        return jsonify({'status': 'healthy', 'timestamp': datetime.utcnow().isoformat()})


    return app


if __name__ == '__main__':
    app = create_app()
    app.run(host='127.0.0.1', port=5001, debug=True)
```

Frontend - frontendappproject.py

```
# Frontend - frontendappproject.py (Main Parts)
# ==================== IMPORTS ====================
import os
import requests
from datetime import datetime
from functools import wraps
from flask import Flask, render_template, request, redirect, url_for, flash, session
from configureproject import Config

app = Flask(__name__,
            template_folder='frontend/templates',
            static_folder='frontend/static')
app.config.from_object(Config)
app.secret_key = Config.SECRET_KEY
API_BASE_URL = 'http://127.0.0.1:5001/api'

# ==================== LOGIN REQUIRED DECORATOR ==================
def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'admin_id' not in session:
            flash('Please login to continue', 'warning')
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated_function
```

```
# ==================== API REQUEST HELPER ====================
def api_request(method, endpoint, data=None):
    url = f"{API_BASE_URL}/{endpoint}"
    cookies = {'session': request.cookies.get('session', '')}
    try:
        if method == 'GET':
            response = requests.get(url, cookies=cookies, timeout=10)
        elif method == 'POST':
            response = requests.post(url, json=data, cookies=cookies, timeout=10)
        elif method == 'PUT':
            response = requests.put(url, json=data, cookies=cookies, timeout=10)
        elif method == 'DELETE':
            response = requests.delete(url, cookies=cookies, timeout=10)
        return response.json() if response.content else {}
    except requests.exceptions.RequestException as e:
        return {'error': str(e)}


# ==================== AUTHENTICATION ROUTES ====================
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        response = api_request('POST', 'auth/login', {'username': username, 'password':
password})
        if 'error' in response:
            flash(response['error'], 'error')
            return render_template('login.html')
        session['admin_id'] = response['admin']['id']
```

```
            session['admin_username'] = response['admin']['username']
            flash('Login successful!', 'success')
            return redirect(url_for('dashboard'))
        return render_template('login.html')

@app.route('/logout')
def logout():
    api_request('POST', 'auth/logout')
    session.clear()
    flash('You have been logged out', 'info')
    return redirect(url_for('login'))

# ==================== DASHBOARD ====================
@app.route('/dashboard')
@login_required
def dashboard():
    stats = api_request('GET', 'dashboard/stats')
    recent_flights = api_request('GET', 'flights?per_page=5')
    recent_messages = api_request('GET', 'messages?per_page=5')
    return render_template('dashboard.html',
                           stats=stats,
                           recent_flights=recent_flights.get('flights', []),
                           recent_messages=recent_messages.get('messages', []))

# ==================== SEND ALERT ====================
@app.route('/send-alert', methods=['GET', 'POST'])
@login_required
def send_alert():
    if request.method == 'POST':
```

```
        alert_data = {
            'flight_id': int(request.form.get('flight_id')),
            'alert_type': request.form.get('alert_type'),
            'message': request.form.get('custom_message'),
            'new_time': request.form.get('new_time'),
            'reason': request.form.get('reason')
        }
        response = api_request('POST', 'alerts/send', alert_data)
        if 'error' in response:
            flash(response['error'], 'error')
        else:
            results = response.get('results', {})
            flash(f"Alert sent! {results.get('successful', 0)} sent, {results.get('failed', 0)} failed",
'success')
            return redirect(url_for('message_history'))
    flights_data = api_request('GET', 'flights?per_page=100')
    alert_types = api_request('GET', 'alerts/types')
    return render_template('send_alert.html',
                           flights=flights_data.get('flights', []),
                           alert_types=alert_types.get('alert_types', []))

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=3000, debug=True)
```

CHAPTER 9
SYSTEM TESTING

SYSTEM TESTING

9.1 Objectives of System Testing

The objective of system testing is to ensure that the SkyAlert: Real-Time Flight Tracking System works correctly and meets the required specifications. System testing verifies that all modules of the system function properly and interact with each other without errors. It helps identify issues in the system and ensures that the application produces accurate results. Another important objective is to confirm that the system can handle user inputs, generate alerts, deliver messages, and store data efficiently. Through system testing, the reliability and performance of the system are validated before deployment.

9.2 Types of Testing

Different types of testing were performed to evaluate the functionality of the system.

Functional Testing: Functional testing checks whether each feature of the system works according to the requirements. It verifies functions such as admin login, alert selection, message generation, and SMS delivery.

Integration Testing: Integration testing ensures that different modules of the system work together correctly. It verifies the interaction between modules such as the alert generation module and the SMS delivery module.

System Testing: System testing evaluates the complete application as a whole. It ensures that the entire system performs correctly and provides the expected output when users interact with it.

Unit Testing: Unit testing involves testing individual components or functions of the system separately. Each module such as the login function, database queries, and SMS sending function is tested independently to ensure it produces the correct output before being integrated with other modules.

Security Testing: Security testing verifies that the system is protected against unauthorized access. It checks whether the admin login authentication works correctly, whether passwords are securely stored using hashing, and whether sensitive passenger data is protected from external threats.

User Acceptance Testing (UAT): User acceptance testing is performed to verify that the system meets the expectations of the end users. It ensures that the admin interface is easy to use, that alerts are sent correctly, and that the overall system functions as intended from the user's perspective.

9.3 Test Results

The testing process showed that all modules of the SkyAlert system function correctly. The admin login module successfully authenticated users, and the alert selection module allowed proper selection of flight alerts. The message generation module created accurate SMS messages based on the selected alerts. The SMS delivery module successfully sent messages to users, and the database module stored the records properly. The results indicate that the system performed as expected and all tests were passed successfully

SMS Delivery Testing

This test verifies that the generated SMS messages are successfully sent to the intended users. The system sends the alert message through the SMS service, and the message is received on the user's mobile phone. The testing confirmed that SMS delivery works properly.

Database Log Testing

This test ensures that all alert information and message records are correctly stored in the system database. The system logs the data after sending alerts so that it can be accessed later if required. The testing confirmed that all records were stored successfully.

9.4 SYSTEM TESTING TABLE

System Testing			
Module	**Expected Result**	**Actual Result**	**Status**
Admin Login	Successful login	Login successful	Pass
Alert Selection	Alert selected	Correct selection	Pass
Message Generation	Proper SMS	Correct SMS	Pass
SMS Delivery	Message sent	SMS received	Pass
Database Log	Data stored	Data stored	Pass

Fig 7 — System Testing Table

CHAPTER 10

RESULTS

RESULT

10.1 Dashboard Overview

1. Dashboard Overview

This interface provides a summary of the flight alert system. It displays the number of registered passengers, available flights, and sent alerts. The dashboard allows administrators to quickly monitor system activity and navigate to other management modules.

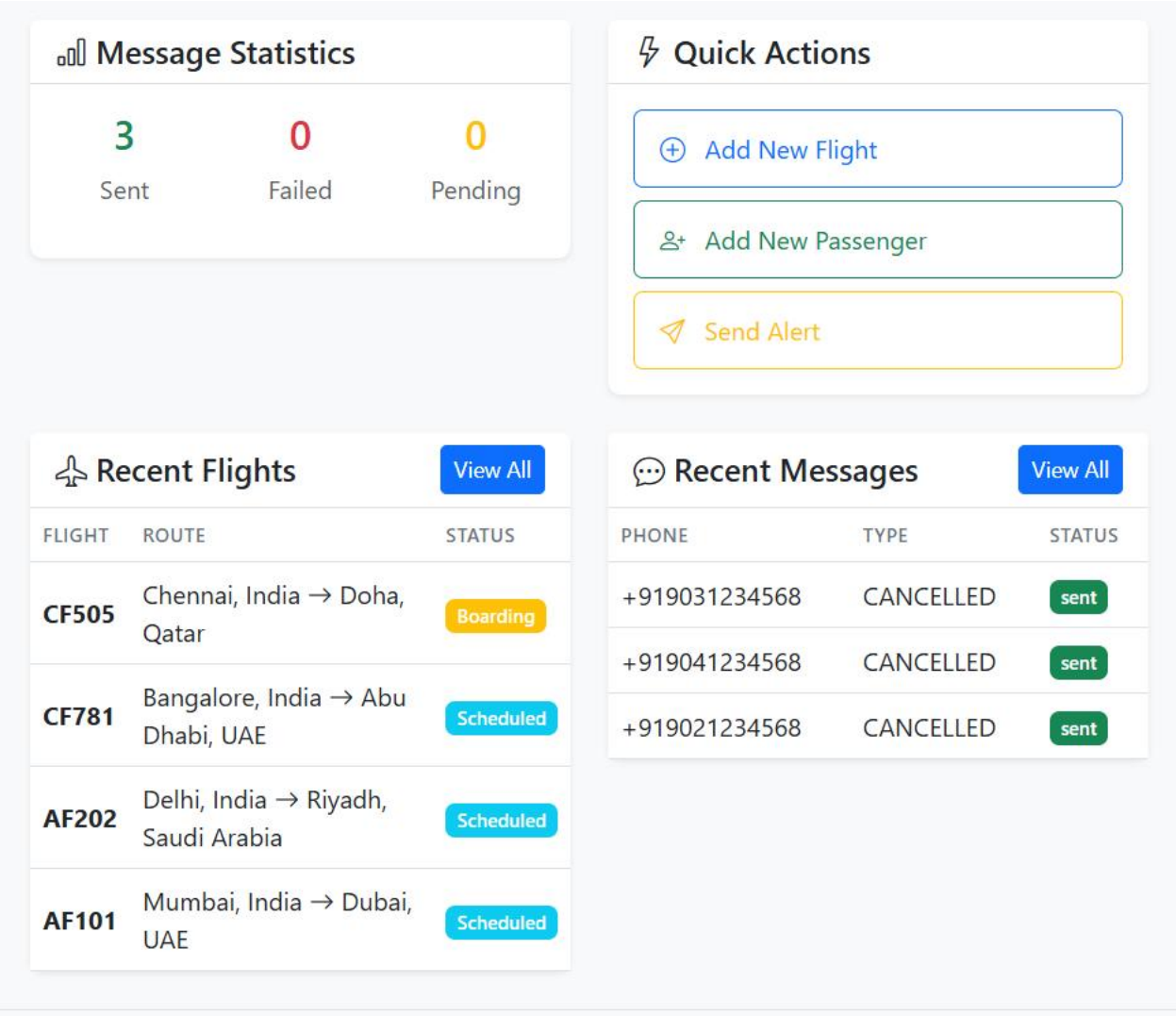


Fig 8 — Dashboard Overview

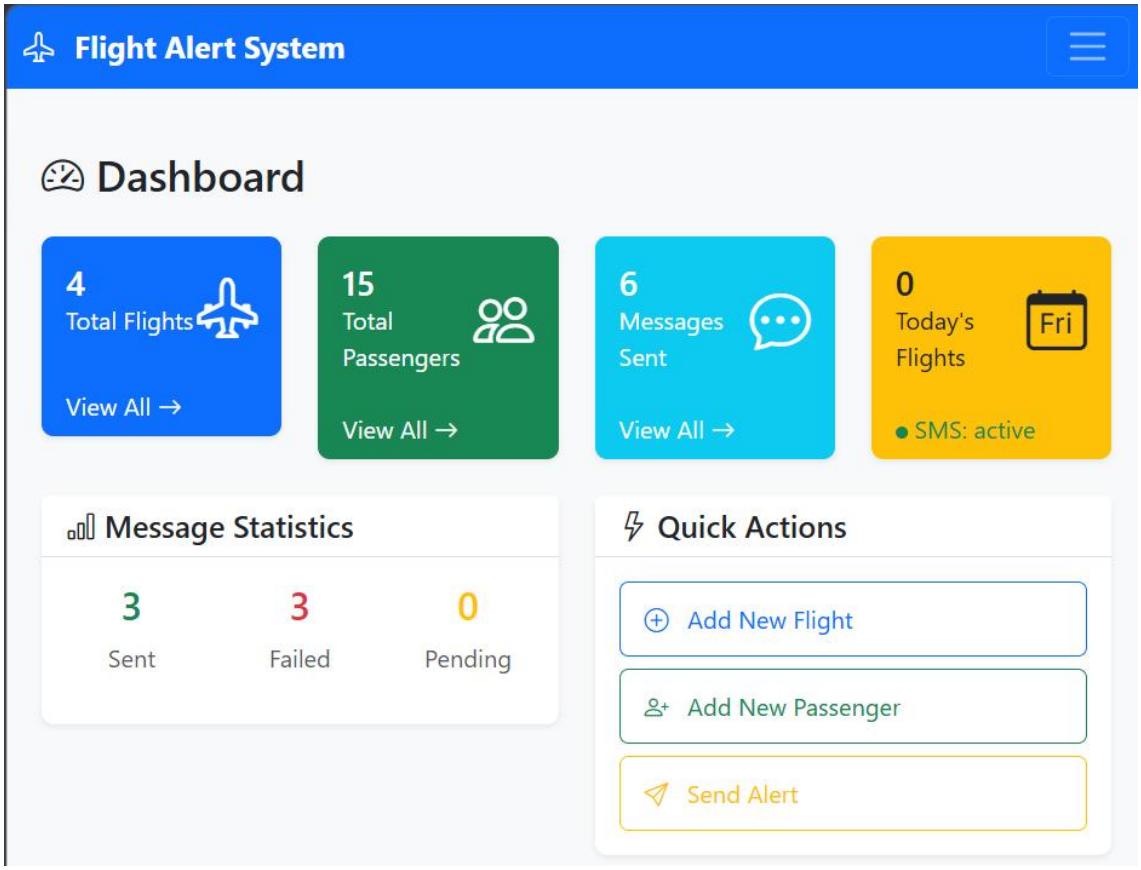


Fig 9 — Dashboard Main View

10.2 passenger management

Passenger Management

This module allows administrators to manage passenger information. Users can view, add, edit, or delete passenger records and configure their notification preferences such as SMS alerts.

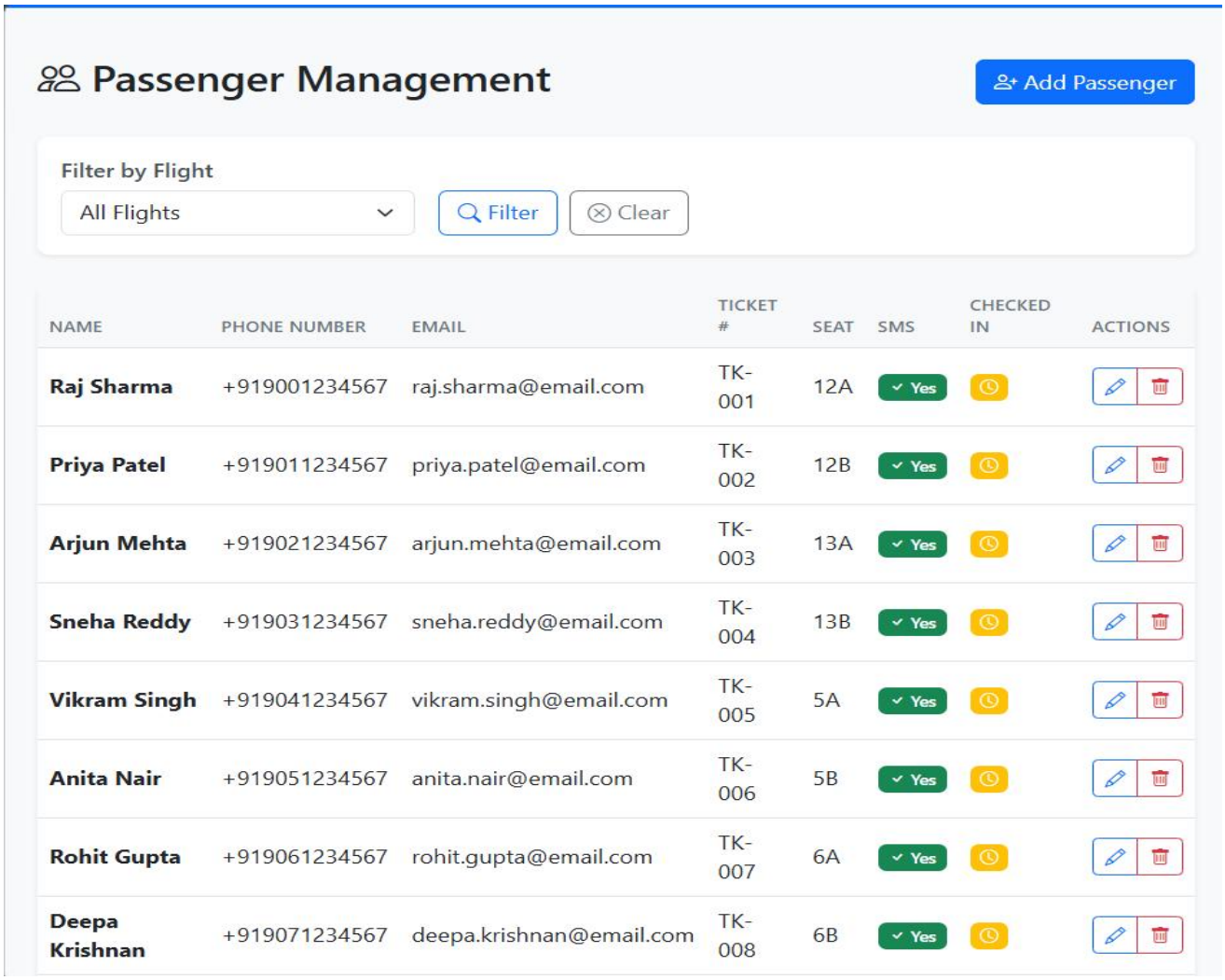


Fig 10— Passenger Management Page

10.3 Flight Management

This interface is used to manage flight details. Administrators can add new flights, update flight schedules, and modify flight status such as delayed, cancelled, or on-time.

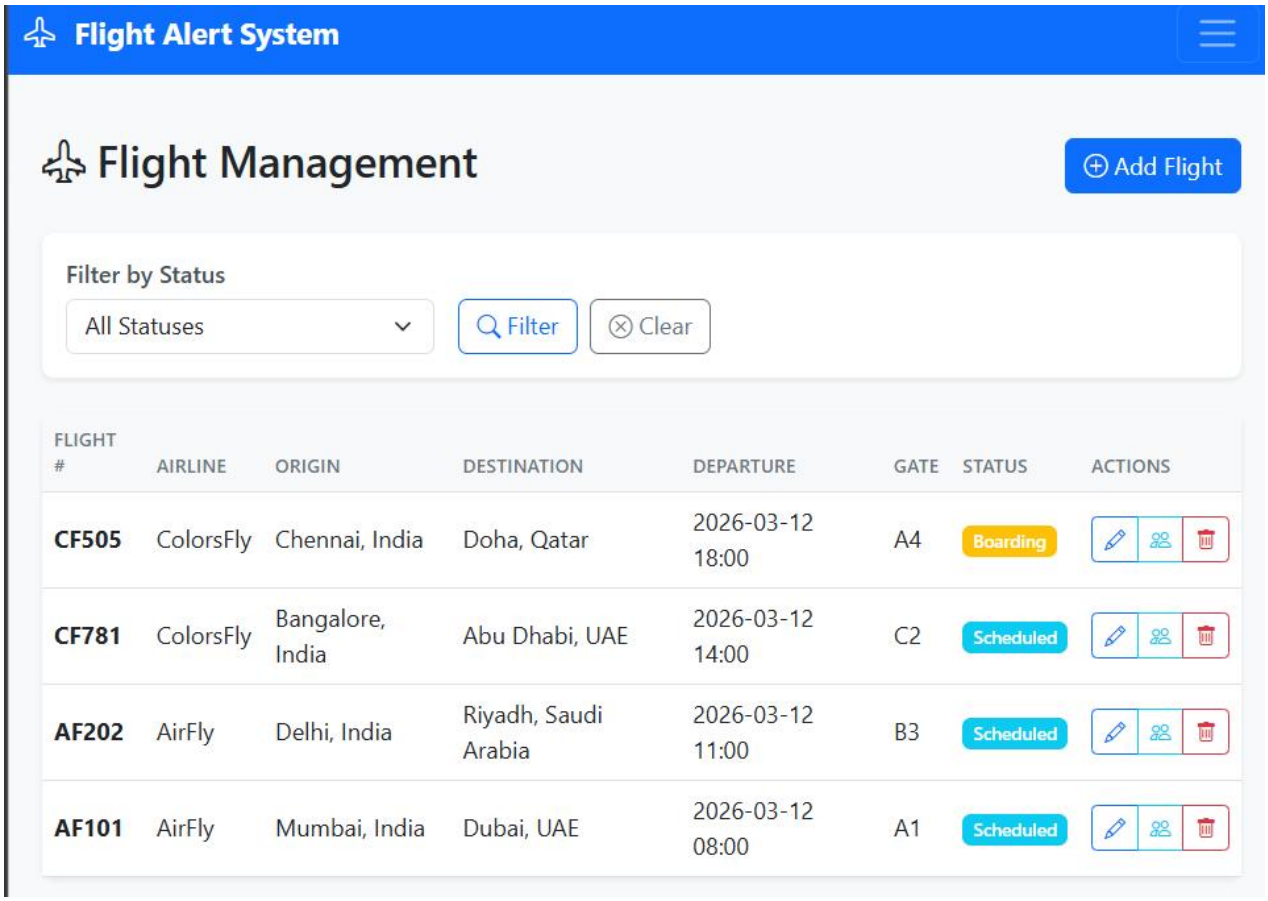


Fig 11 — Flight Management Page

10.4 Sending Flight Alerts

This module enables administrators to send SMS alerts to passengers registered for a specific flight. The administrator selects the flight, alert type, and optionally adds a custom message before sending the notification.

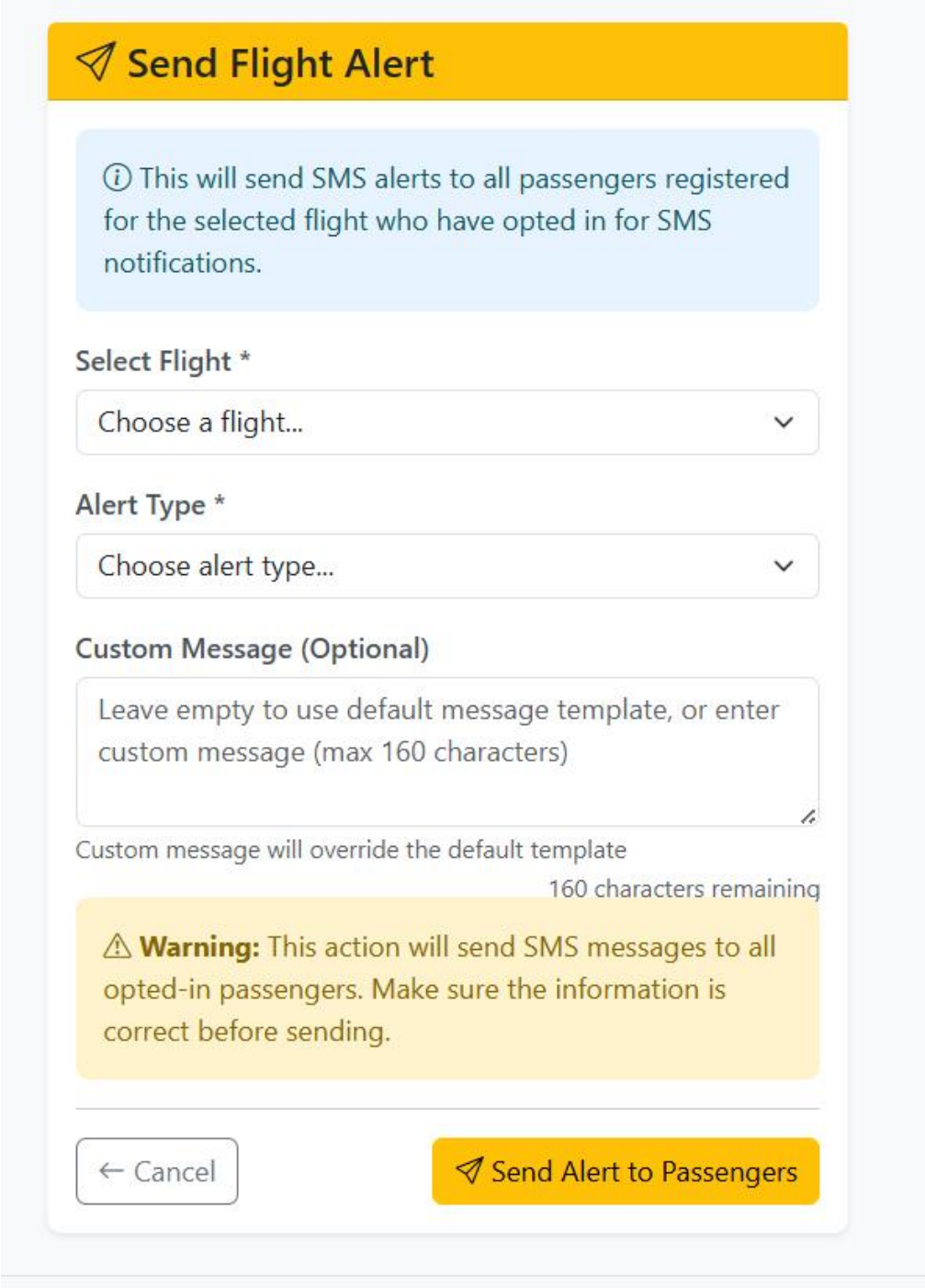


Fig 12 — Send Flight Alert Page

10.5Message History Tracking

The system stores all previously sent notifications. This page allows administrators to view the alert history, including the message content, flight details, delivery status, and timestamp.

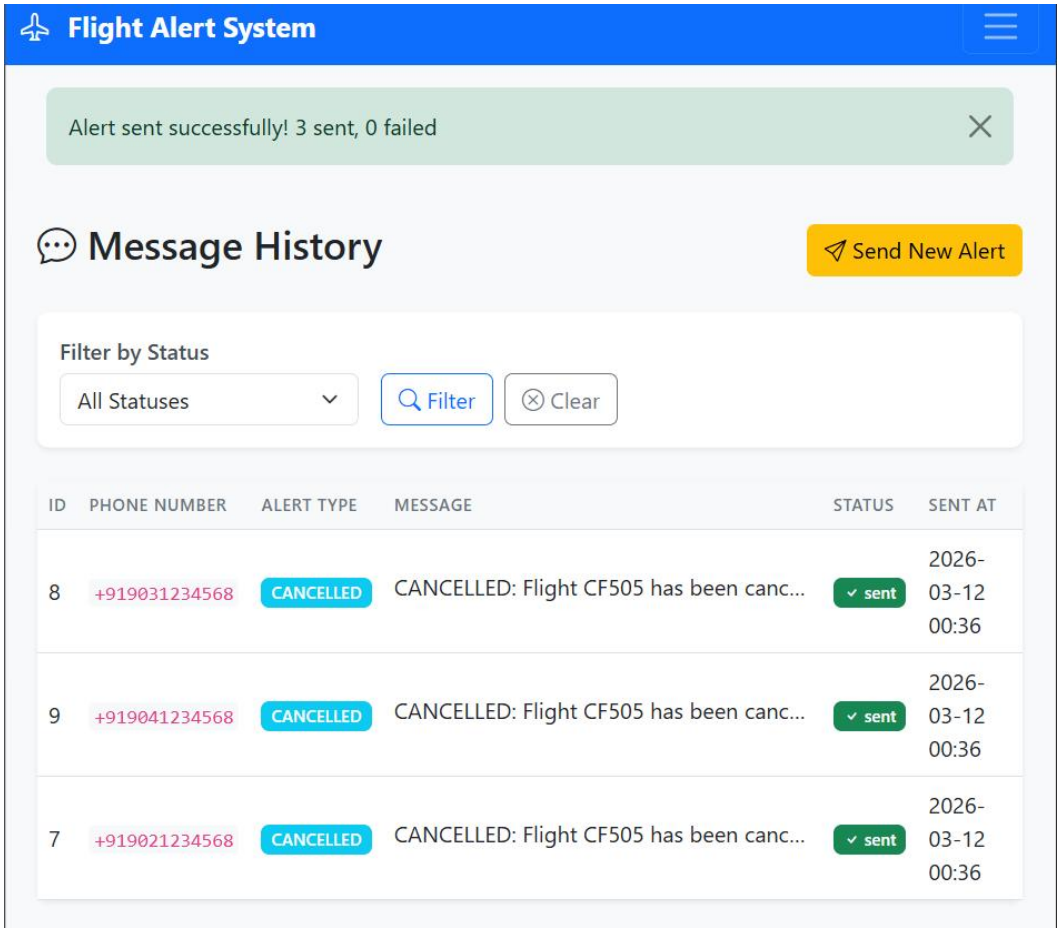


Fig 13 — Message History Page

10.6 SMS Alert Delivery

Once the alert is sent, passengers receive an SMS notification on their mobile devices. The message contains important flight updates such as delays, cancellations, or boarding announcements.

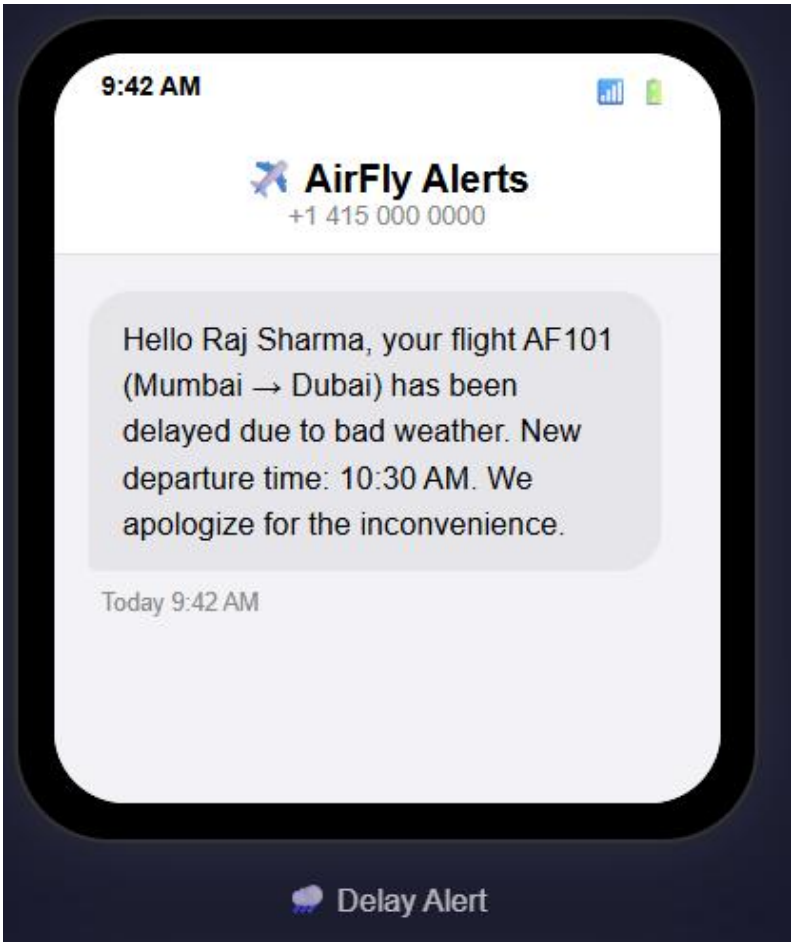


Fig 14 — Delay Alert SMS

10.7 Example Alert Notifications

Sample alert messages demonstrate how passengers receive flight status updates on their mobile phones. These notifications ensure passengers stay informed in real time.

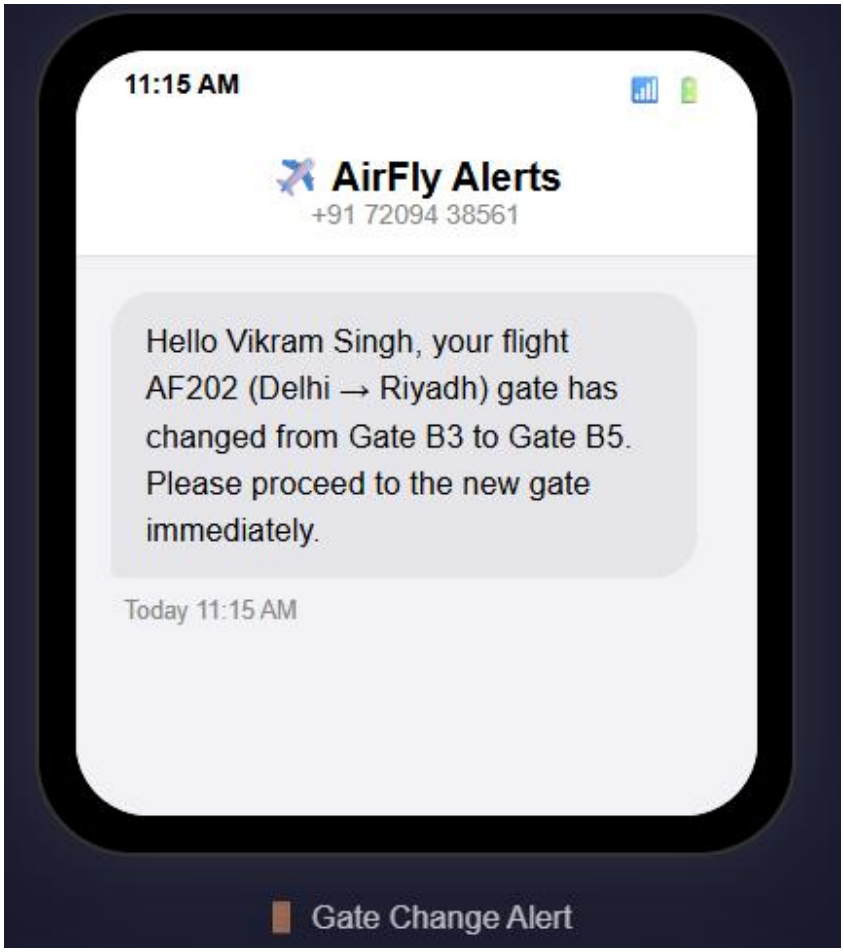


Fig 15 — Gate Change Alert SMS

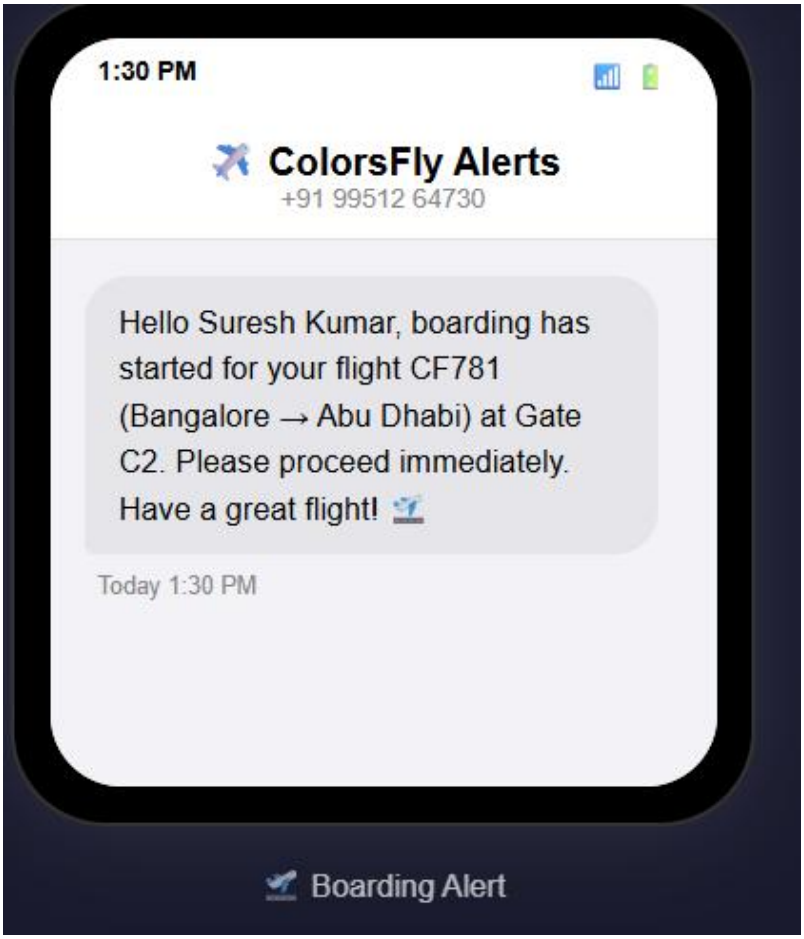


Fig 16 — Boarding Alert SMS

CHAPTER 11

CONCLUSION

AND

FUTURE ENHANCEMENT

CONCLUSION AND FUTURE ENHANCEMENT

11.1 CONCLUSION

The Flight Alert System using SMS is a practical and efficient solution designed to improve communication between airports and passengers. In modern air travel, timely information is very important for passengers to manage their journey smoothly. Flight schedules may change due to weather conditions, technical issues, or operational reasons. Without proper communication, passengers may experience confusion, delays, or even miss important announcements. This project aims to address these issues by providing a reliable system that automatically sends flight updates directly to passengers through SMS messages.

The system is designed to deliver real-time notifications such as flight delays, gate changes, boarding announcements, and other important updates. Instead of depending only on airport display boards or public announcements, passengers receive the information directly on their mobile phones. This makes the communication process faster, clearer, and more convenient for travelers. Since almost everyone carries a mobile phone, SMS becomes a simple and effective way to ensure that important flight information reaches passengers instantly.

One of the major advantages of this system is that it does not rely on internet connectivity. In many crowded airport environments, internet access may be slow, unstable, or unavailable. However, SMS technology works even on basic mobile networks, making it a highly reliable communication method. This ensures that passengers receive important alerts even if they do not have access to smartphones or mobile data services. As a result, the system becomes more inclusive and accessible for a wider range of users.

The project is developed using modern and widely used technologies. Python is used as the main programming language to handle system operations and automate the process of sending notifications. MySQL is used to store and manage important data such as passenger details, flight information, and notification records. An SMS service such as Twilio is integrated into the system to deliver text messages quickly and reliably to passengers' mobile numbers. These technologies work together to create a simple, scalable, and efficient system.

Overall, the Flight Alert System using SMS improves passenger convenience and enhances communication efficiency within airport environments. By reducing confusion and providing timely updates, the system helps passengers stay informed and better prepared for their flights. This project demonstrates how technology can be effectively applied to solve real-world problems in the aviation industry. It also highlights the importance of automated communication systems in improving the travel experience for passengers and supporting better airport management.

7.2 FUTURE ENHANCEMENT

1. **Real-Time Flight Tracking:** The system can be upgraded to provide live tracking of flights so passengers receive exact arrival and departure updates.

2. **Multi-Language Support:** SMS alerts can be sent in different languages based on passenger preference, making it helpful for international travellers.

3. **Voice Call Notifications:** In addition to SMS, automated voice calls can be introduced for urgent announcements like final boarding calls.

4. **Mobile Application Development:** A dedicated mobile application can be developed to provide additional features along with SMS alerts.

5. **AI-Based Delay Prediction:** Artificial Intelligence can be used to predict flight delays based on weather conditions and air traffic data.

6. **Two-Way Communication:** Passengers can be allowed to reply to SMS messages to confirm boarding or request information.

7. **Baggage and Seat Information Alerts:** The system can send updates about baggage belt numbers and seat changes.

8. **Emergency Alert System:** Special alerts can be sent instantly during emergencies, cancellations, or sudden schedule changes.

9. **Integration with Airport Management Systems:** The system can be directly connected with airport databases for faster and automatic updates.

10. **Advanced SMS Tracking:** Using advanced features from services like Twilio, message delivery status and scheduling can be improved.

REFERENCE

REFERENCES

Rossum, G. van. (2020). **Python Programming Language Documentation.** Python Software Foundation. Available online: https://docs.python.org/3/

1. Grinberg, M. (2018). **Flask Web Development: Developing Web Applications with Python.** O'Reilly Media.

2. Oracle Corporation. (2021). **MySQL 8.0 Reference Manual.** Available online: https://dev.mysql.com/doc/

3. Twilio Inc. (2022). **Twilio SMS API Documentation.** Available online: https://www.twilio.com/docs/sms

4. W3C. (2018). **HTML5 Specification.** World Wide Web Consortium. Available online: https://www.w3.org/TR/html5/

5. W3C. (2019). **Cascading Style Sheets Level 3 (CSS3) Specification.** Available online: https://www.w3.org/Style/CSS/

6. Pressman, R. S. (2014). **Software Engineering: A Practitioner's Approach.** McGraw-Hill Education.

7. Sommerville, I. (2016). **Software Engineering (10th Edition).** Pearson Education.

8. IEEE. (2012). **IEEE Standard for Software and System Test Documentation.** IEEE Std 829.

9. Replit Inc. (2023). **Replit Online Integrated Development Environment.** Available online: https://replit.com/